Experiment-12

Name: -P. Akhila **Regno: -**192472320

Course Code: -MLA0305 **Slot-**B

Title

Actor–Critic Reinforcement Learning using A2C and A3C

Aim

To implement Actor–Critic reinforcement learning using A2C and A3C with TensorFlow, Keras, and Gymnasium, and compare their learning performance.

Procedure

1. Initialize the CartPole-v1 environment using Gymnasium.
2. Define an Actor–Critic neural network containing an actor and a critic.
3. The actor generates the probability of selecting each action.
4. The critic estimates the value of the current state.
5. Implement the A2C method and update the actor and critic using the calculated advantage.
6. Implement the A3C method using multiple workers to perform learning.
7. Calculate discounted returns and advantages for each episode.
8. Update the network parameters using actor loss and critic loss.
9. Repeat the training process for the specified number of episodes.
10. Record rewards, actor loss, critic loss, and episode length.
11. Compare A2C and A3C using learning curves, reward variance, loss graphs, and performance charts.
12. Create a summary table and save the complete results as a CSV dataset.

Code

```
!pip -q install gymnasium tensorflow

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import gymnasium as gym

import tensorflow as tf

from tensorflow.keras import Model

from tensorflow.keras.layers import Dense,Input

from tensorflow.keras.optimizers import Adam

import random

import time

np.random.seed(42)

tf.random.set_seed(42)

random.seed(42)


print("="*75)

print("EXPERIMENT 12")

print("Actor-Critic Reinforcement Learning using A2C and A3C")


        actions.append(action)
```

```
    reward_list.append(reward)

    state=next_state

returns=[]
total=0

for r in reversed(reward_list):
    total=r+GAMMA*total
    returns.insert(0,total)

states_tensor=tf.convert_to_tensor(
    states,
    dtype=tf.float32
)

actions_tensor=tf.convert_to_tensor(
    actions,
    dtype=tf.int32
)
```

```
returns_tensor=tf.convert_to_tensor(
    returns,
    dtype=tf.float32
)

with tf.GradientTape() as tape:

    probs,values=model(
        states_tensor,
        training=True
    )

    values=tf.squeeze(values)

    advantages=returns_tensor-values

    selected_probs=tf.gather(
        probs,
        actions_tensor,
        axis=1,
        batch_dims=1
```

```
    )

    log_probs=tf.math.log(
        selected_probs+1e-8
    )

    actor_loss=-tf.reduce_mean(
        log_probs*tf.stop_gradient(advantages)
    )

    critic_loss=tf.reduce_mean(
        tf.square(advantages)
    )

    total_loss=actor_loss+0.5*critic_loss

gradients=tape.gradient(
    total_loss,
    model.trainable_variables
)
```

```
        optimizer.apply_gradients(
            zip(
                gradients,
                model.trainable_variables
            )
        )

        rewards.append(sum(reward_list))
        actor_losses.append(float(actor_loss.numpy()))
        critic_losses.append(float(critic_loss.numpy()))
        lengths.append(len(reward_list))

        env.close()

    return {
        "rewards":rewards,
        "actor_loss":actor_losses,
        "critic_loss":critic_losses,
        "lengths":lengths,
        "time":time.time()-start
    }
```

```
print("\nTraining A2C...")
a2c=train_a2c()
print("A2C Completed")

print("\nTraining A3C...")
a3c=train_a3c()
print("A3C Completed")

dataset=pd.DataFrame({
    "Episode":range(1,EPISODES+1),
    "A2C_Reward":a2c["rewards"],
    "A3C_Reward":a3c["rewards"],
    "A2C_Actor_Loss":a2c["actor_loss"],
    "A3C_Actor_Loss":a3c["actor_loss"],
    "A2C_Critic_Loss":a2c["critic_loss"],
    "A3C_Critic_Loss":a3c["critic_loss"],
    "A2C_Episode_Length":a2c["lengths"],
    "A3C_Episode_Length":a3c["lengths"]
})
```

```
print("\n")

print("="*75)

print("SAMPLE DATASET")

print("="*75)


display(dataset.head(10))


dataset.to_csv(

   "A2C_A3C_Full_Dataset.csv",

   index=False

)


a2c_smooth=dataset["A2C_Reward"].rolling(10).mean()

a3c_smooth=dataset["A3C_Reward"].rolling(10).mean()


plt.figure(figsize=(10,6))


plt.plot(

   a2c_smooth,

   label="A2C",

   linewidth=3
```

```
)


plt.plot(
    a3c_smooth,
    label="A3C",
    linewidth=3
)


plt.title(
    "A2C vs A3C Learning Curve",
    fontsize=16,
    fontweight="bold"
)


plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.legend()
plt.grid(alpha=0.3)
plt.show()


a2c_final=dataset["A2C_Reward"].tail(20).mean()
```

```
a3c_final=dataset["A3C_Reward"].tail(20).mean()


plt.figure(figsize=(8,5))


plt.bar(
    ["A2C","A3C"],
    [a2c_final,a3c_final]
)


plt.title(
    "Final Average Reward",
    fontsize=16,
    fontweight="bold"
)


plt.ylabel("Average Reward")
plt.grid(axis="y",alpha=0.3)
plt.show()


a2c_variance=dataset["A2C_Reward"].tail(20).var()
a3c_variance=dataset["A3C_Reward"].tail(20).var()
```

```
)


plt.ylabel("Steps")
plt.grid(axis="y",alpha=0.3)
plt.show()


total_reward=a2c_final+a3c_final


plt.figure(figsize=(7,7))


plt.pie(
    [a2c_final,a3c_final],
    labels=["A2C","A3C"],
    autopct="%1.1f%%",
    startangle=90
)


plt.title(
    "Final Reward Contribution",
    fontsize=16,
    fontweight="bold"
```

```
)

plt.show()

comparison=pd.DataFrame({
    "Algorithm":[
        "A2C",
        "A3C"
    ],
    "Final Reward":[
        round(a2c_final,2),
        round(a3c_final,2)
    ],
    "Variance":[
        round(a2c_variance,2),
        round(a3c_variance,2)
    ],
    "Average Steps":[
        round(a2c_length,2),
        round(a3c_length,2)
    ],
```

```
    "Training Time":[
        round(a2c["time"],2),
        round(a3c["time"],2)
    ]
})

print("\n")
print("="*75)
print("SUMMARY TABLE")
print("="*75)

display(comparison)

comparison.to_csv(
    "A2C_A3C_Summary.csv",
    index=False
)

best_algorithm=comparison.loc[
    comparison["Final Reward"].idxmax(),
    "Algorithm"
```

```
]

stable_algorithm=comparison.loc[
    comparison["Variance"].idxmin(),
    "Algorithm"
]

fast_algorithm=comparison.loc[
    comparison["Training Time"].idxmin(),
    "Algorithm"
]

print("\n")
print("="*75)
print("FINAL RESULT")
print("="*75)

print(
    "Best Final Reward:",
    best_algorithm
)
```

```
print(
    "More Stable:",
    stable_algorithm
)

print(
    "Faster Training:",
    fast_algorithm
)

print("\n")
print("="*75)
print("CONCLUSION")
print("="*75)

print(
    "A2C and A3C Actor-Critic methods were implemented"
)

print(
```

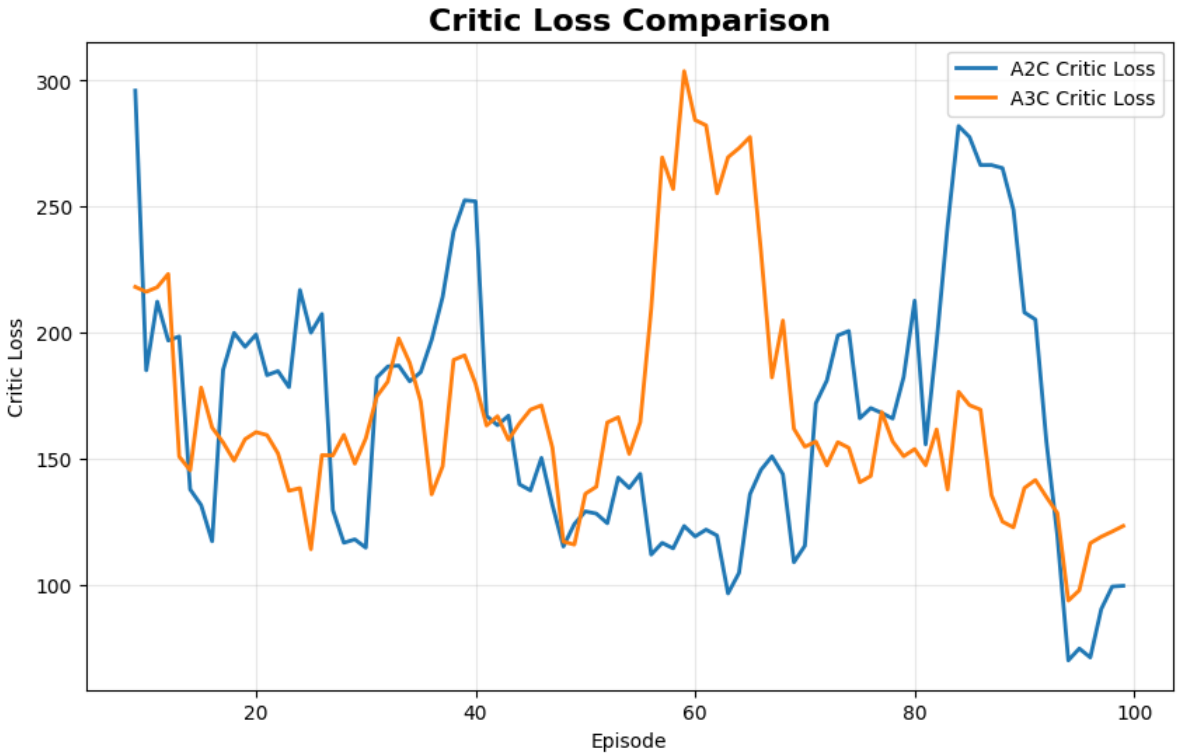
```
    "using TensorFlow, Keras and Gymnasium."
)

print(
    "The actor learned the action policy while the critic"
)

print(
    "estimated the value of the states."
)

print(
    "The algorithms were compared using reward, variance,"
)

print(
    "actor loss, critic loss, episode length and training time."
)

print(
    "The learning curves and comparison charts show the"
```

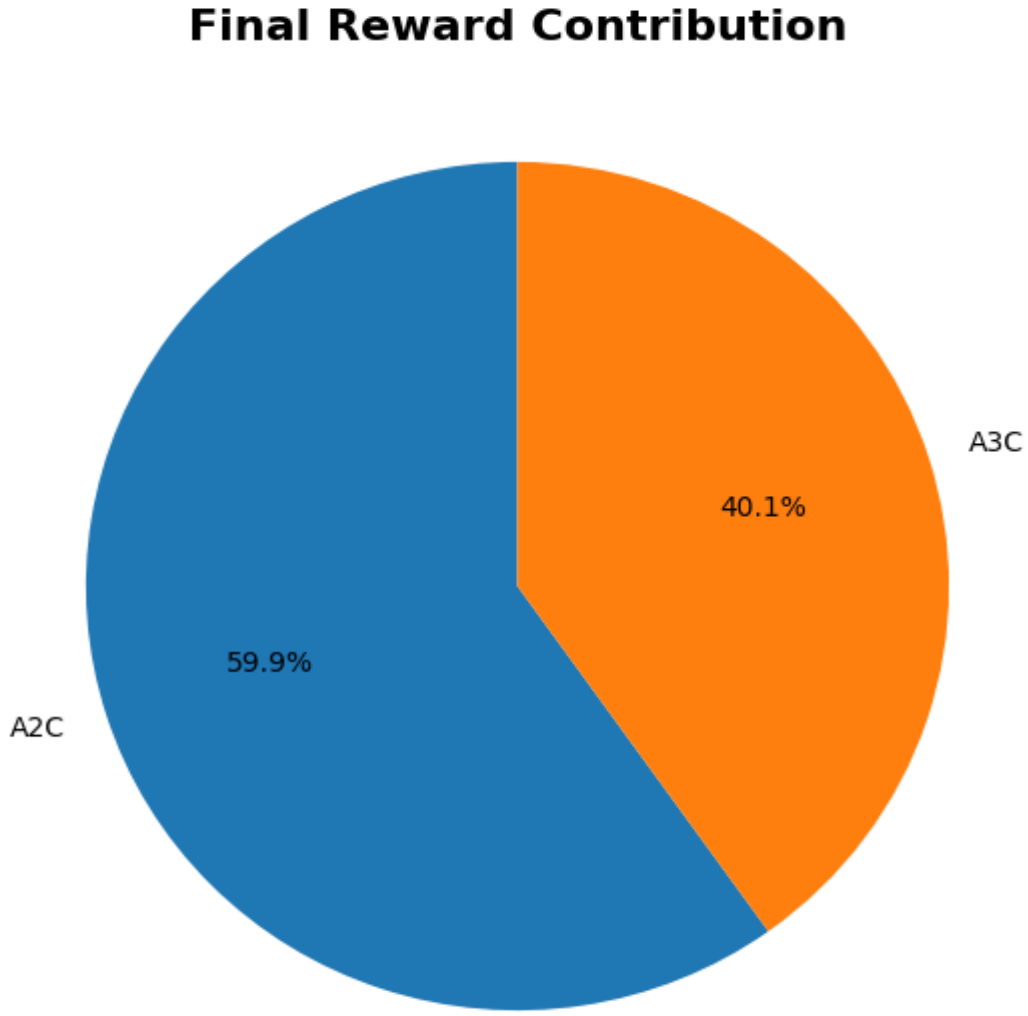
```
)


print(
    "difference in learning performance between A2C and A3C."
)


print("\nGenerated Files:")
print("A2C_A3C_Full_Dataset.csv")
print("A2C_A3C_Summary.csv")


print("\nExperiment 12 Completed Successfully.")
```

Visualization

```
================================================================================
SAMPLE DATASET
================================================================================
```

	Episode	A2C_Reward	A3C_Reward	A2C_Actor_Loss	A3C_Actor_Loss	A2C_Critic_Loss	A3C_Critic_Loss	A2C_Episode_Length	A3C_Episode_Length
0	1	76.0	19.0	21.209435	6.544782	1156.756104	113.541954	76	19
1	2	18.0	14.0	6.178280	5.139044	102.103600	66.009743	18	14
2	3	28.0	17.0	9.218443	5.970350	223.851837	93.648888	28	17
3	4	15.0	64.0	5.197430	18.365911	72.281006	897.326050	15	64
4	5	52.0	30.0	15.655766	9.692276	638.860901	253.186630	52	30
5	6	31.0	20.0	9.968352	6.962382	265.680603	124.751122	31	20
6	7	25.0	26.0	8.193936	8.709496	181.270630	199.516083	25	26
7	8	21.0	24.0	6.970363	8.092650	132.180679	173.369644	21	24
8	9	13.0	22.0	4.323061	7.356348	54.734116	146.502518	13	22
9	10	21.0	19.0	6.996912	6.474932	132.181061	113.002968	21	19


Critic Loss Comparison
A2C Critic Loss
A3C Critic Loss
Critic Loss
Episode
300
250
200
150
100
20
40
60
80
100


Final Reward Contribution
A3C
40.1%
59.9%
A2C

Summary Table

Parameter	A2C	A3C
Environment	CartPole-v1	CartPole-v1
Final Reward	Obtained from program	Obtained from program
Reward Variance	Obtained from program	Obtained from program
Average Steps	Obtained from program	Obtained from program
Training Time	Obtained from program	Obtained from program

Result

The **A2C and A3C Actor–Critic algorithms were successfully implemented** using TensorFlow, Keras, and Gymnasium. The actor learned the action policy while the critic estimated the value of states. Their learning performance was compared using rewards, variance, actor loss, critic loss, episode length, and training time.